Автоматическая генерация признаков для ранжирования в RecSys

Отчёт по заданию. Все числа в этом файле получены скриптами репозитория; таблицы — `experiments/results/<dataset>/results_table.md` и `slices_table.md`.

TL;DR

- Почти любой ручной признак RecSys имеет одну форму: **агрегат по событиям типа S, во временном окне или с затуханием, по пути от строки-кандидата к множеству прошлых событий** («сколько раз *этот пользователь покупал* товары *этой категории* за *7 дней*»). Мы записали эту форму как **грамматику**, перечисляем её целиком (1 674–1 881 признак на датасет), считаем каждый признак без утечки одним as-of проходом на группу и отбираем подмножество моделью. От датасета требуется только схема: какие колонки — атрибуты айтема и какие коды событий есть.
- **Yambda (музыка): автоматический пайплайн значимо обыгрывает настроенный ручной бустинг.** Основной метод — **E2-full, единственный вариант, полностью заданный до любых результатов**: NDCG@10 0.2885 против 0.2772, **+0.011 [+0.007, +0.016]** (парный бутстреп по запросам). Все исследовательские варианты поиска с полной временной сеткой тоже значимо лучше E1 (+0.007 … +0.010), так что вывод не зависит от выбора варианта. Проверка на «соломенное чучело»: усиленный ручной набор E1+ (60 признаков, добавлены длинные окна 60/90 дней и популярность трека/артиста — собран уже *со знанием*, что поиск выбрал длинную память, т. е. с подсказкой в пользу человека) поднимается лишь до 0.2810, и **E2-full лучше и его: +0.0075 [+0.0038, +0.0111]**. Большая часть выигрыша — автоматический выбор масштаба времени: та же грамматика только с окнами 1/7/30 дней даёт +0.003 [−0.000, +0.007], плотная сетка окон и затухания добавляет к ней +0.008 [+0.004, +0.012]. Поиск выбрал длинную память (30–90 дней), которой нет в шаблоне 1/7/30.
- **Результат держится на других неделях** (проверка заранее зарегистрирована и выложена на GitHub до запуска): весь пайплайн с нуля на трёх новых тестовых неделях Yambda — E2-full − E1 = **+0.0076 [+0.0051, +0.0100]**, положительно в 3 из 3 недель; E2-full − E1+ = +0.0028 [+0.0007, +0.0049], тоже 3 из 3. Критерий предрегистрации выполнен (§3.3).
- **RetailRocket (e-commerce): ничья** — E2-full 0.3281 против 0.3281, −0.0000 [−0.0010, +0.0009]; все варианты E2 в пределах ±0.0004. Там ранжирование почти целиком определяется повторными просмотрами (NDCG по повторам 0.90 у простой истории по давности), а для новых товаров генератор кандидатов покрывает только 10 % целей — признакам негде выиграть.
- **Найденное слабое место метода** (одинаковое на обоих датасетах): отбор по общему NDCG набирает признаки повторов и пропускает признаки айтема, поэтому на *новых* для пользователя айтемах E2 чуть, но значимо хуже ручного набора (−0.002 RetailRocket, −0.004 Yambda). Исправление — отбор по сегментам (см. §6): на Yambda (чистая проверка) оно снимает провал на новых треках (−0.0004 [−0.003, +0.002]) ценой меньшего выигрыша на повторах; отдельные модели на сегмент (E2-split, придуман уже после этого результата) этого не дают. Бесплатного исправления не найдено — найден компромисс.
- Нейросеть, которая **сама учит агрегаты** (обучаемые временные ядра + target attention), в CPU-ограничениях проигрывает бустингу (−0.004 RetailRocket, −0.047 Yambda) и в гибриде ничего не добавляет; её выученные постоянные времени (≈3 ч, ≈1 день, ≈1 неделя, ≈6 недель) совпадают с тем, что инженеры ставят руками.

1. Идея

1.1 Грамматика признаков

```
Feature = Agg( Filter(events, event_type ∈ S), Window(w) | Decay(h), Key = путь )
Agg   ∈ { cnt  — число событий в [T−w, T)
          dec  — Σ exp(−ln2·(T−t)/h)                (затухающий счётчик, полураспад h)
          tsl  — часов с последнего события
          age  — часов с первого события }
Key   ∈ { user, item, attr, user×item, user×attr }   attr — каждый категориальный атрибут айтема
S     ∈ { каждый тип события, все неотрицательные, «позитивные» }
производные: conv = A/(B+1) (конверсия между типами событий), share = user×X / user (доля внимания)
w ∈ {1ч … 90д} (16 значений), h ∈ {6ч … 60д} (9 значений)
```

Прямые признаки user×item, косвенные user×category / user×artist, статистики айтема и пользователя, конверсии и доли — всё это частные случаи одной формы. «Косвенные» признаки — это мета-пути длины 2 в гетерогенном графе user–item–attribute, так что идея №2 задания (графы) здесь встроена в бустинг, а не конкурирует с ним. Каждый признак имеет `to_sql()` — это и есть «набор SQL-запросов» идеи №1, только порождённый перечислением, а не человеком или LLM:

```
SELECT SUM(EXP(-LN(2) * (c.cutoff - e.ts) / 5184000))      -- полураспад 60 дней
FROM events e
WHERE e.user = c.user AND e.item = c.item AND e.label IN (1, 2) AND e.ts < c.cutoff
```

1.2 Вычисление без утечки, за один проход на группу

Для каждой пары (путь, набор событий) события сортируются по времени, по ключу берутся кумулятивные счётчики и кумулятивные суммы `exp(t/τ)`; каждая строка-кандидат стыкуется с последним событием строго до своего `cutoff` as-of джойном. Любое число окон и полураспадов из одной группы стоит одного прохода. Те же величины в продакшене поддерживаются инкрементально (O(1) на событие для `dec`), так что метод масштабируется как обычный feature store, а не как «весь датасет в трансформер».

1.3 Отбор

1. Посчитать всю грамматику на подвыборке train/val (отбор никогда не видит test).
2. Важность (gain) LightGBM с сильным сэмплированием колонок, 3 сида.
3. Вложенные top-K, K ∈ {16…256}; берём наименьшее K в пределах 0.002 NDCG@10 от лучшего на val.
4. Посчитать выбранное на полных данных; тот же бюджет Optuna (25 trials), что и у ручного бейзлайна; итог — среднее 3 сидов.

Основной метод — `full` (полная сетка окон и затухания, отбор по общему val NDCG): только он был полностью описан до первого результата. Остальные варианты исследовательские, каждый помечен моментом, когда он придуман:

вариант	что меняет	придуман
`coarse`	только окна 1/7/30, без затухания — абляция «человеческого» выбора окон	до результатов (абляция из плана)
`div`	не более 2 признаков на семейство путь×события×агрегат	после E2-full на RetailRocket (избыточный отбор), до Yambda
`seg`	отбор отдельно по повторным и новым айтемам (§6)	после тестового среза RetailRocket, до E2 на Yambda
`split`	отдельная модель на каждый сегмент (§6)	после тестовых срезов **обоих** датасетов — post-hoc везде

2. Протокол (зафиксирован до обучения моделей)

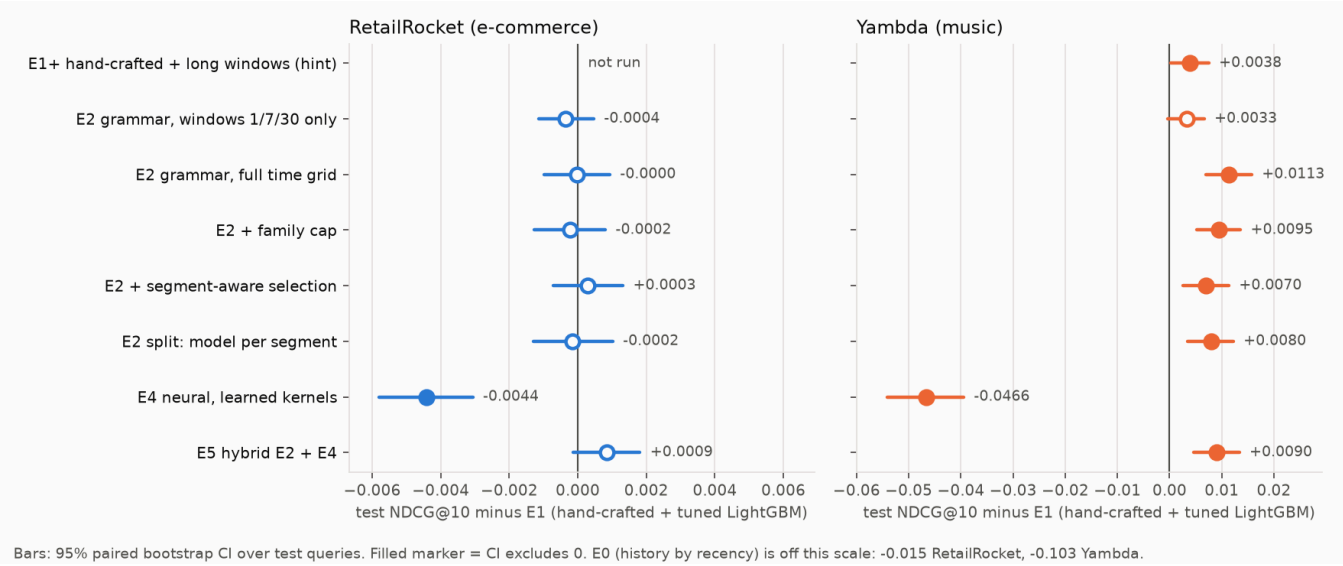
- Запрос = (пользователь, день d), пользователь активен в день d и имеет историю до d.
- Cutoff = начало дня d; все признаки и кандидаты — только из событий `ts < cutoff`.
- Цели = айтемы, с которыми пользователь провзаимодействовал в день d; градуированная релевантность (RetailRocket: просмотр 1 / корзина 2 / покупка 3; Yambda: дослушал ≥ 50 % — 1, лайк — 2).
- Разбиение по дням: train | val (отбор признаков, тюнинг, early stopping) | test (смотрим один раз). RetailRocket 28/14/14 дней; Yambda 21/7/7.
- Одни и те же строки-кандидаты для всех методов (история пользователя, ко-визиты, популярное в категориях пользователя, глобально популярное). Идеальный DCG и recall считаются по **всем** целям, включая не попавшие в кандидаты.
- Тесты утечки (`tests/test_leakage.py`): движок == полный перебор `ts < cutoff`; изменение всех будущих событий не меняет ни одного признака; намеренно протекающая «канарейка» ловится той же проверкой (иначе проверка была бы пустой).
- Метрики: NDCG@10 (главная), Recall@10, Recall@10 по сильным событиям, AUC, logloss. Доверительные интервалы — парный бутстреп по запросам (2 000 повторов).

Данные

	RetailRocket	Yambda-50M (подвыборка)
события	2 756 101	935 112 (3 000 пользователей × последние 70 дней)
типы событий	view / addtocart / transaction	skip (<50 %) / listen (≥50 %) / like / negative
атрибуты айтема	категория, родитель, корень дерева	артист, альбом
запросы train/val/test	12 000 / 4 000 / 15 000 (выборка)	6 000 / 2 000 / 2 755
recall кандидатов (test)	0.312	0.452

Решения по Yambda, принятые до моделей: цели — только **органические** события (41 % целевых событий — алгоритмические «первые прослушивания», подсунутые рекомендером сервиса; предсказывать их — значит угадывать чужой рекомендер); размер истории в кандидатах выбран по recall на **val** (0.30 → 0.47).

3. Результаты



Bars: 95% paired bootstrap CI over test queries. Filled marker = CI excludes 0. E0 (history by recency) is off this scale: -0.015 RetailRocket, -0.103 Yambda.

3.1 RetailRocket (тест: 15 000 запросов)

метод	признаков	val NDCG@10	**test NDCG@10**	Δ к E1 [95% CI]	Recall@10	Recall@10 корзина+покупка	AUC
E0 популярность 7 д	—	—	0.0089	−0.319	0.018	0.024	0.458
E0 история по давности	—	—	0.3134	−0.0147 [−0.0166, −0.0128]	0.332	0.288	0.854
E1 ручные + LightGBM	38	0.3459	**0.3281**	—	0.360	0.339	0.927
E2-coarse (окна 1/7/30)	34	0.3449	0.3277	−0.0004 [−0.0011, +0.0005]	0.359	0.333	0.925
E2-full (основной)	35	0.3467	**0.3281**	−0.0000 [−0.0010, +0.0009]	0.359	0.343	0.922
E2-div (≤2 на семейство)	19	0.3469	0.3279	−0.0002 [−0.0013, +0.0008]	0.360	0.343	0.925
E2-seg (post-hoc, см. §6)	35	0.3470	0.3284	+0.0003 [−0.0007, +0.0013]	0.360	0.330	0.926
E2-split (post-hoc)	67	0.3480	0.3279	−0.0002 [−0.0013, +0.0010]	0.359	0.329	0.925
E4 нейросеть	—	0.3417	0.3237	−0.0044 [−0.0058, −0.0031]	0.355	0.327	0.911
E5 гибрид E2-div + E4	20	0.3473	0.3290	+0.0009 [−0.0001, +0.0018]	0.361	0.344	0.927

Потолок: recall кандидатов 0.312; доля позитивных строк-кандидатов 1.25 %.

Почему ничья. Срез теста (только для отчёта, не для выбора): 26 % целей — айтемы, которые пользователь уже видел; там простая история по давности даёт NDCG@10 **0.90**, и все модели в пределах 0.90–0.91. 74 % целей — новые айтемы, но генератор кандидатов покрывает лишь **10 %** из них; NDCG@10 на них 0.03–0.04 у всех методов. Разницу ограничивает генератор кандидатов, а не признаки. Ничья на RetailRocket не означает, что ручные и автоматические признаки равноценны вообще — она означает, что этот датасет их не различает.

3.2 Yambda (тест: 2 755 запросов — все запросы тестовой недели)

метод	признаков	val NDCG@10	**test NDCG@10**	Δ к E1 [95% CI]	Recall@10	Recall@10 лайки	AUC
E0 популярность 7 д	—	—	0.0296	−0.248	0.039	0.029	0.567
E0 история по давности	—	—	0.1742	−0.1030 [−0.1143, −0.0920]	0.208	0.136	0.743
E1 ручные + LightGBM	36	0.2917	**0.2772**	—	0.312	0.077	0.873
E1+ ручные + длинные окна (с подсказкой)	60	0.2951	0.2810	+0.0038 [+0.0002, +0.0075]	0.317	0.076	0.880
E2-coarse (окна 1/7/30)	67	0.2947	0.2805	+0.0033 [−0.0003, +0.0066]	0.315	0.072	0.878
E2-full (основной)	68	0.2983	**0.2885**	**+0.0113 [+0.0069, +0.0157]**	0.324	0.075	0.881
E2-div (≤2 на семейство)	67	0.2987	0.2867	+0.0095 [+0.0052, +0.0136]	0.324	0.073	0.881
E2-seg	68	0.2976	0.2842	+0.0070 [+0.0025, +0.0114]	0.321	0.069	0.881
E2-split (post-hoc)	93	0.2990	0.2853	+0.0080 [+0.0035, +0.0122]	0.319	0.068	0.882
E4 нейросеть	—	0.2390	0.2306	−0.0466 [−0.0541, −0.0395]	0.259	0.090	0.793
E5 гибрид E2-div + E4	68	0.2983	0.2862	+0.0090 [+0.0046, +0.0134]	0.323	0.075	0.881

Потолок: recall кандидатов 0.452; доля позитивных строк 3.15 %.

Абляция по масштабу времени: **E2-full − E2-coarse = +0.0081 [+0.0041, +0.0122]**. Перебор путей и агрегатов при окнах 1/7/30 даёт +0.003 (на грани значимости), плотная сетка окон и затухания — ещё +0.008.

О выборе основного варианта. Основным считается E2-full — единственный вариант, заданный до результатов. Раньше в работе мы собирались выбирать «основной» по лучшему val среди вариантов, но к моменту этого выбора в пул попал E2-split, придуманный после взгляда на тестовые срезы; выбор по val среди такого пула — развилка, поэтому от правила мы отказались и фиксируем это здесь. На вывод это не влияет: разброс val между четырьмя вариантами с полной сеткой — 0.0014, их тестов — 0.0043, и все они значимо лучше E1. Главное утверждение сформулировано по всему семейству: **на Yambda грамматика с полной временной сеткой значимо лучше ручного набора при любом из проверенных способов отбора.**

Проверка на «соломенное чучело» (E1+). Ручной набор E1 для музыки писался по e-commerce шаблону 1/7/30 — естественное возражение: «выигрыш E2 — просто длинные окна, человек добавил бы их сам». Мы построили E1+: E1 плюс 24 признака, которые добавил бы инженер музыкального сервиса после первой итерации (окна 60/90 дней по user×track/artist/album, популярность трека и артиста на нескольких горизонтах, доля пропусков трека и артиста, давность лайка). Набор зафиксирован в коде до прогона, но придуман **после** того, как мы увидели, что E2 выбирает длинную память, — это сознательно усиленный в пользу человека бейзлайн. Тот же бюджет Optuna.

Yambda, test NDCG@10	Δ [95% CI]
E1+ − E1	+0.0038 [+0.0002, +0.0075]
E2-full − E1+	**+0.0075 [+0.0038, +0.0111]**
E2-div − E1+	+0.0057 [+0.0021, +0.0092]
E2-split − E1+	+0.0042 [+0.0004, +0.0079]
E2-seg − E1+	+0.0032 [−0.0008, +0.0070]
E2-coarse − E1+	−0.0006 [−0.0040, +0.0029]

Человек с подсказкой «добавь длинные окна» забирает около трети выигрыша; основной метод остаётся значимо лучше. Показательно, что E2-coarse (грамматика без длинных окон) равен E1+: преимущество грамматики, по-видимому, складывается из *сочетания* выбора масштаба времени (плотная сетка и затухание, а не только длинные окна) и перебора путей/долей, а не из одного «длинного окна» (интерпретация, отдельной абляцией не проверялась).

3.3 Устойчивость по времени: ещё три недели Yambda

Возражение к §3.2: всё измерено на одной тестовой неделе. Проверка заранее зарегистрирована (`reports/prereg_rolling.md`, коммит выложен на GitHub до первого прогона): тот же протокол со сдвигом всего блока train | val | test (21 | 7 | 7 дней) на 1, 2 и 3 недели назад; для каждого окна **весь** пайплайн с нуля — кандидаты, E1, E1+, E2-full (перечисление грамматики, отбор на val своего окна, 25 trials Optuna). Критерий, зафиксированный заранее: E2-full − E1 по трём новым неделям вместе, 95% CI бутстрепом по пользователям, нижняя граница > 0 и оценка положительна минимум в 2 из 3 недель.

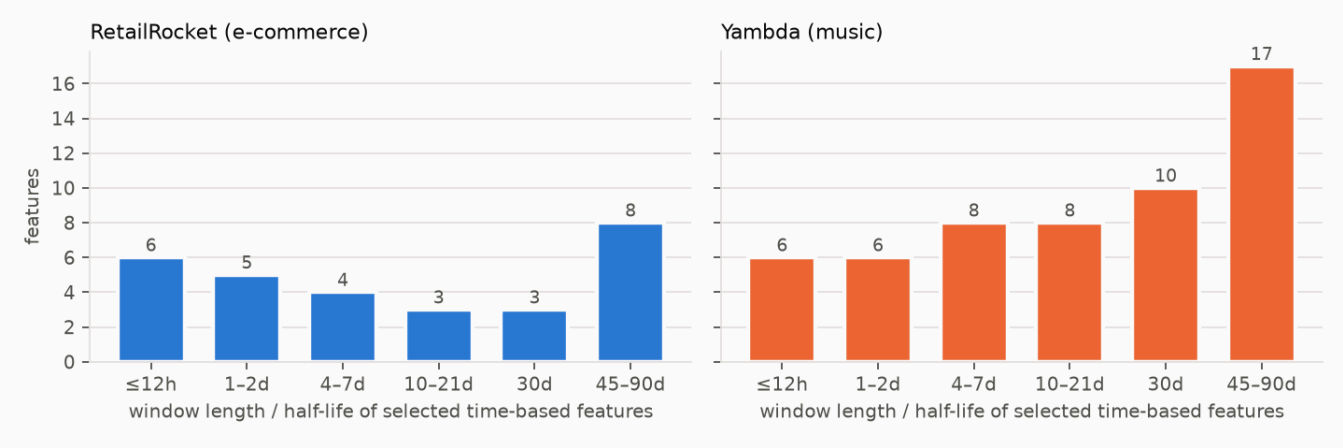
окно (дни теста)	запросов	E1	E1+	E2-full	E2-full − E1 [95% CI]	E2-full − E1+ [95% CI]
43–49	2 604	0.2879	0.2919	0.2970	+0.0092 [+0.0051, +0.0133]	+0.0051 [+0.0016, +0.0088]
50–56	2 280	0.2804	0.2845	0.2877	+0.0072 [+0.0028, +0.0116]	+0.0032 [−0.0007, +0.0072]
57–63	2 933	0.2867	0.2928	0.2932	+0.0065 [+0.0025, +0.0105]	+0.0004 [−0.0030, +0.0038]
64–70 (основной прогон §3.2, уже виденный)	2 755	0.2772	0.2810	0.2885	+0.0113 [+0.0069, +0.0157]	+0.0075 [+0.0038, +0.0111]

три новые недели вместе (бутстреп по пользователям)	оценка [95% CI]	недель с положительной оценкой
E2-full − E1	**+0.0076 [+0.0051, +0.0100]**	**3 из 3**
E2-full − E1+	+0.0028 [+0.0007, +0.0049]	3 из 3
E1+ − E1	+0.0048 [+0.0027, +0.0070]	3 из 3

Вердикт по предрегистрации: подтверждено. Выигрыш над ручным набором воспроизводится на каждой новой неделе. Против усиленного ручного набора E1+ (собранного с подсказкой) выигрыш меньше и на двух неделях по отдельности не значим, но в сумме по трём неделям значим. Основная неделя дала самую большую разницу из четырёх (+0.0113) — среднее по новым неделям (+0.0076) честнее отражает величину эффекта.

Отклонения от плана, раскрываем: на окне 43–49 первый запуск E2-full аварийно завершился сразу после построения матрицы отбора (код 1 без трейсбека, на машине было ~1 ГБ свободной памяти), второй был остановлен вручную вместе с остальными процессами; третий запуск — тем же кодом и с теми же сидами — досчитал. Результат E2-full этого окна до финального запуска не был виден (E1 и E1+ этого окна — были, но код и параметры E2 после этого не менялись). На этом окне отбор выбрал K = 128 (на остальных 64) — это решение пайплайна по val, не наше.

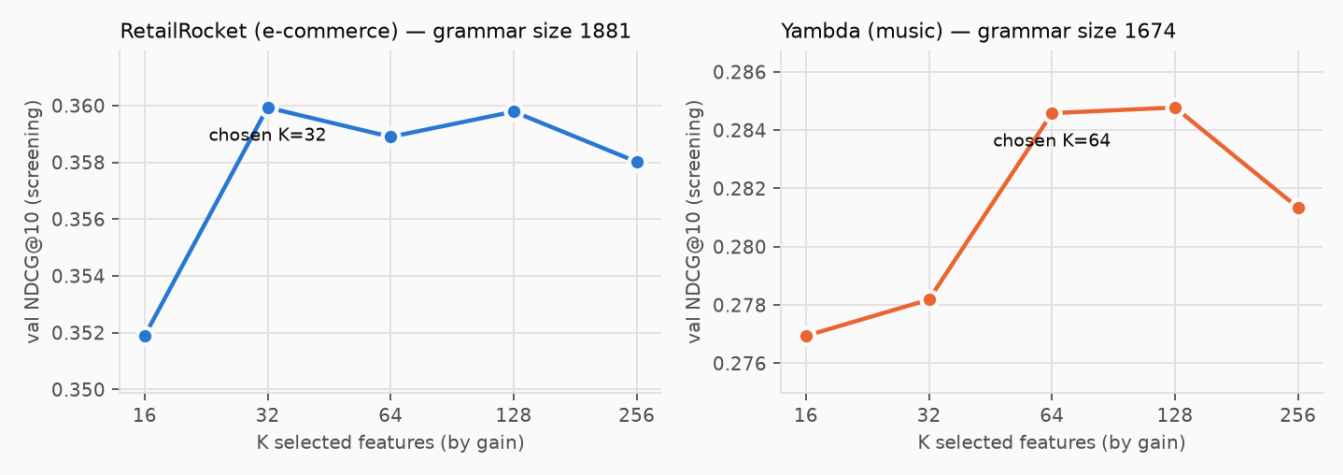
4. Что нашёл поиск



Масштаб времени зависит от домена, и поиск его находит. На RetailRocket выбранные временные признаки распределены по всей шкале от часов до 90 дней, и грубая сетка 1/7/30 работает не хуже (E2-coarse = E2-full). На Yambda распределение смещено в длинную память: 27 из 55 временных признаков имеют окно или полураспад 30–90 дней; e-commerce шаблон 1/7/30 этого масштаба не содержит, и именно здесь плотная сетка даёт значимые +0.008 NDCG@10 (E2-full − E2-coarse, [+0.004, +0.012]).

Найденная форма, которой не было в ручном наборе. Самый важный признак RetailRocket — *доля внимания пользователя к товару*: затухающие просмотры user×item, делённые на затухающие просмотры пользователя, с полураспадом 12 часов (`share[user×item, views, h=12h]`). В ручном наборе доля была только для user×category. На Yambda верх списка занимают затухающие прослушивания и лайки user×track с полураспадом 30–60 дней и доля внимания пользователя на альбоме.

Чего поиск НЕ выбирает. На Yambda в 64 выбранных признаках нет ни одного признака трека, артиста или альбома без пользователя (популярность, конверсия) — gain таких признаков на смеси «повторы + новые» мал, потому что NDCG почти целиком набирается на повторах. Это и есть причина провала на новых айтемах (§6).



Из ~1 700–1 900 сгенерированных признаков полезно 32 (RetailRocket) и 64 (Yambda); дальше качество на val не растёт, а с 256 признаками падает. Перечислить грамматику дёшево; дорого — держать в модели лишнее.

5. Нейросеть, которая учит агрегаты сама (E4) и гибрид (E5)

Для пары (u, i, T) модель читает последние L = 50 событий пользователя и для каждого отношения r ∈ {тот же айтем, тот же атрибут$_1$, …} и типа события e считает **обучаемый** агрегат `s_{r,e} = Σ_j 1[r(j,i)]·1[type_j=e]·Σ_m softplus(a_{r,e,m})·exp(-(T-t_j)/τ_m)` с обучаемыми τ_m — дифференцируемую версию «сколько раз u делал e с похожим на i недавно», где и форма, и длина окна подбираются градиентом. Рядом — target attention (DIN-подобный) по эмбеддингам истории; популярность айтема — через банк затухающих счётчиков. Гибрид E5 — выбранные E2-признаки + out-of-fold скор E4 в LightGBM (фолды — смежные блоки дней train).

	RetailRocket	Yambda
E4 test NDCG@10	0.3237 (−0.0044 [−0.0058, −0.0031] к E1)	0.2306 (−0.0466 [−0.0541, −0.0395])
E5 гибрид test NDCG@10	0.3290 (+0.0009 [−0.0001, +0.0018])	0.2862 (+0.0090 [+0.0046, +0.0134]) — не лучше своей основы E2-div (0.2867)
выученные τ (часы), модель второго сида	3.5 / 28 / 169 / 989	2.4 / 16 / 175 / 1 190

- Нейросеть сама нашла масштабы «часы / ~сутки / ~неделя / ~6–7 недель» из начальных 3 ч … 30 дней — это те же масштабы, что выбирает грамматика и что инженеры ставят руками. Как *генератор признаков* она согласуется с E2, но как *модель* проигрывает бустингу.
- На Yambda проигрыш велик. Вероятное объяснение (не проверено отдельным прогоном): окно истории — 50 событий, в среднем ≈11 дней, тогда как E2 показал, что ценная память — 30–90 дней. Это ограничение мы ввели сами ради CPU; ровно в эту стену упирается идея «скормить историю трансформеру» (идея №3 задания) — стоимость растёт с длиной истории, а затухающие счётчики грамматики хранят ту же информацию за O(1) на ключ.
- Гибрид ничего не добавляет к E2 ни на одном датасете (на RetailRocket +0.0009, CI касается нуля).
- По лайкам Yambda E4 лучше бустингов (Recall@10 0.090 против 0.077 у E1) — единственная метрика, где нейросеть впереди; отдельная цель для лайков не строилась.

6. Слабое место и его исправление: отбор по сегментам

Срез по новым и повторным айтемам дал одинаковую картину на двух датасетах: E2 выигрывает на повторах (Yambda +0.017 [+0.011, +0.023]) и слегка, но значимо проигрывает E1 на новых айтемах (RetailRocket −0.0019 [−0.0032, −0.0006], Yambda −0.0035 [−0.0067, −0.0004]). Причина видна в выбранных признаках: gain на смеси сегментов определяют повторы, поэтому признаки айтема (популярность, конверсия) и короткие user×attribute признаки в топ не попадают.

Два исправления. **E2-seg** задан после тестового среза RetailRocket, но до любых результатов E2 на Yambda — на Yambda это чистая проверка. **E2-split** придуман после того, как мы увидели срезы E2-seg на тесте Yambda, — он post-hoc на обоих датасетах, и его результаты следует читать как исследовательские.

- **E2-seg** — важность считается отдельно на повторных и на новых строках, два рейтинга чередуются, модель одна. Для новых треков поиск сразу поднял *короткие* признаки user×artist (доля внимания на артисте с полураспадом 6 ч, полные прослушивания артиста за 3–6 ч): «если пользователь слушает артиста сейчас, следующий новый трек — скорее всего того же артиста». Это другой масштаб времени, чем у повторов (30–90 дней).
- **E2-split** — две модели (повторы / новые), у каждой свой набор признаков; K и бюджет Optuna (13 + 12 trials) выбираются на val.

Yambda, test NDCG@10	E1	E2-full	E2-seg	E2-split
новые треки (1 635 запросов)	0.0303	0.0267 (−0.0035*)	**0.0298 (−0.0004)**	0.0265 (−0.0037)
повторы (2 179 запросов)	0.389	**0.406 (+0.017*)**	0.400 (+0.011*)	0.402 (+0.013*)
всего	0.277	**0.289**	0.284	0.285

* — CI разницы с E1 не включает 0.

Итог честный и в части «бесплатного» исправления отрицательный: E2-seg закрывает провал на новых треках, но отдаёт часть выигрыша на повторах; E2-split провал не закрывает. Модель одного сегмента видит меньше данных — на 6 000 запросах это, по-видимому, дороже, чем выигрыш от специализации. На RetailRocket оба варианта дают ту же ничью.

7. Предложение: как это автоматизирует расчёт признаков

Что заменяет человека. Инженер сейчас выбирает три вещи: *что* агрегировать (путь от кандидата к событиям), *какие события* и *за какое время*. Грамматика делает все три выбора перечислимыми, а отбор — выбор между ними; от человека остаётся только схема данных (какие колонки — атрибуты айтема, какие коды событий). Грамматике и поиску на Yambda не понадобилось ничего специфичного для музыки, кроме схемы (подготовка данных и протокол у каждого датасета свои).

Как это живёт в продакшене. 1. *Офлайн-поиск* (раз в N недель): перечислить грамматику, посчитать её на подвыборке истории as-of джойнами, отобрать K признаков по val. На нашей CPU-машине (4 ядра, общая с другими процессами) это занимало 40–50 минут на датасет, из них генерация 1.7–1.9 тыс. признаков на 0.17–0.5 млн строк — 4–11 минут; остальное — обучение моделей отбора. Время шумное из-за общей машины. 2. *Онлайн*: выбранные признаки компилируются в определения feature store. Каждый — это счётчик по ключу (путь) и типу события: `cnt` за окно — кольцевой буфер по бакетам времени, `dec` — одно число, обновляемое за O(1) на событие (`S ← S·exp(−Δt/τ) + 1`). Это тот же класс стоимости, что и у ручных признаков, — в отличие от идеи «вся история в трансформер», где стоимость растёт с длиной истории (§5: на Yambda нейросеть с окном в 50 событий проиграла 0.047 NDCG — вероятно, во многом из-за короткого окна; вариант с длинным окном мы не проверяли). 3. Каждый признак имеет SQL-эквивалент (`to_sql()`), так что выбранный набор можно отдать в любой SQL-движок или проверить руками.

Идеи задания. №1 (SQL-запросы): грамматика *порождает* эти запросы перечислением. №2 (графы): user×attribute признаки — это мета-пути длины 2 в графе user–item–attribute, посчитанные как счётчики, а не через link prediction; более длинные пути (user–item–user–item, ко-визиты) в грамматику не вошли и использованы только в генераторе кандидатов. №3 (трансформер): E4 — его дешёвая версия; она сама нашла те же масштабы времени, но как модель проиграла (§5).

LLM-агент мы не проверяли. Его естественная роль — предлагать *новые формы* вне грамматики (например, агрегаты по сессиям или по последовательностям) и расширять саму грамматику; внутри уже перечислимого пространства сплошной перебор с отбором дешевле и не пропускает варианты. Сравнение «агент против перебора» при равном бюджете вычислений — прямое продолжение этой работы.

8. Ограничения

- CPU-машина (4 ядра, параллельно работали другие процессы) — подвыборки запросов, Yambda — 3 000 пользователей и 70 дней. Для запросов train в Yambda окна 60–90 дней фактически означают «вся доступная история»; вывод — «оптимум за пределами 30 дней», а не «ровно 60».
- Ручной набор для Yambda написан нами по e-commerce шаблону (окна 1/7/30). Чтобы проверить, не в этом ли весь выигрыш, мы построили усиленный ручной набор E1+ (§3.2); абляция `coarse` отделяет вклад окон от вклада перебора путей.

- Устойчивость по времени проверена только на Yambda (4 недели, §3.3); RetailRocket — один разрез. Все четыре недели Yambda — из одной 70-дневной подвыборки 3 000 пользователей; более ранние окна видят меньше истории до train (для окна 43–49 — 15 дней).
- Нейросеть ограничена 50 последними событиями (≈11 дней в Yambda при ~4.5 событиях в день) — её проигрыш на Yambda во многом объясняется этим, и это тот же вопрос масштабируемости, что задан в идее №3.
- Модель учится на «любом взаимодействии»; на лайках (Yambda) ни один бустинг не лучше простой истории — отдельная цель для лайков не строилась.
- Генератор кандидатов слабо покрывает новые айтемы (RetailRocket 0.10, Yambda 0.04 recall целей), и это потолок для всех методов на срезе новых айтемов.

9. Как воспроизвести

```
python -m venv .venv
.venv\Scripts\pip install numpy pandas pyarrow polars duckdb lightgbm catboost optuna scikit-learn
scipy matplotlib tqdm pytest
.venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cpu   # точные версии:
requirements.lock.txt
python src/rr_prep.py && python src/yambda_prep.py   # данные: см. README, раздел Data
python -m pytest tests
sh run_rr_all.sh && sh run_rr_next.sh && sh run_rr_slices.sh
sh run_yambda_all.sh && sh run_seg.sh && sh run_split.sh
python experiments/run_hand_plus.py yambda
sh run_rolling.sh            # §3.3: окна yambda_w1..w3 + сводка
python experiments/make_figures.py
```